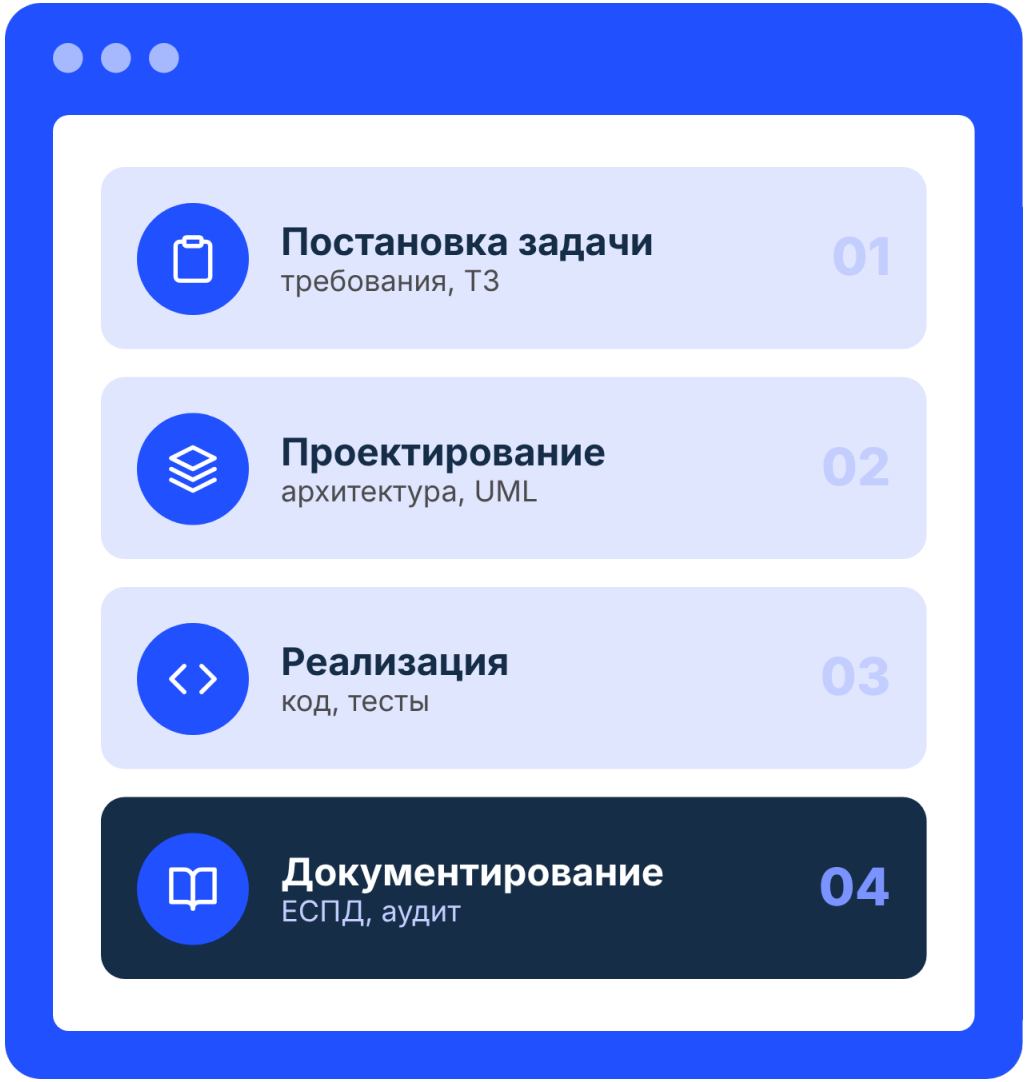
Лекция 4

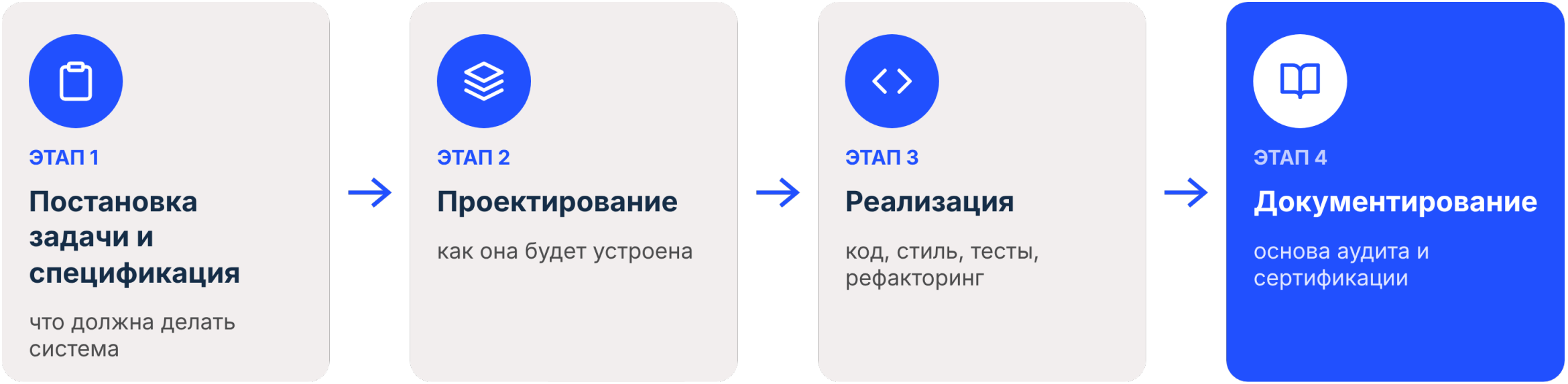
Основные этапы технологии программирования

Постановка задачи и спецификация, проектирование и реализация, документирование программ



Ключевые этапы, определяющие успех проекта

Нужны не только технические навыки, но и умение работать с требованиями, моделями и стандартами.



Анализ требований

Первый и важнейший этап создания ПО.

Ошибки на этом этапе обходятся дороже всего

Цель — понять, что должна делать система, для кого и с какими ограничениями

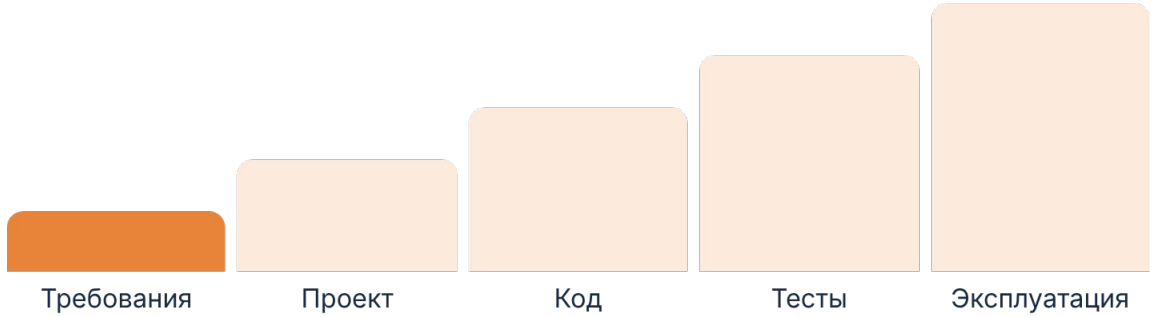
Что должна делать система?

Какие задачи решает?

Кто пользователи?

Какие ограничения?

Ошибка в требованиях проходит по всем этапам:



чем позже найдена — тем **дороже** исправление

Виды требований

ЧТО

Функциональные

что система должна делать

«Генерировать пароль длиной от 8 до 64 символов»

КАК

Нефункциональные

производительность, надёжность, безопасность, удобство

«Генерация пароля — не дольше 0,1 секунды»

ЗАЧЕМ

Бизнес-требования

зачем создаётся система, какая выгода

ДЛЯ ЧЕГО

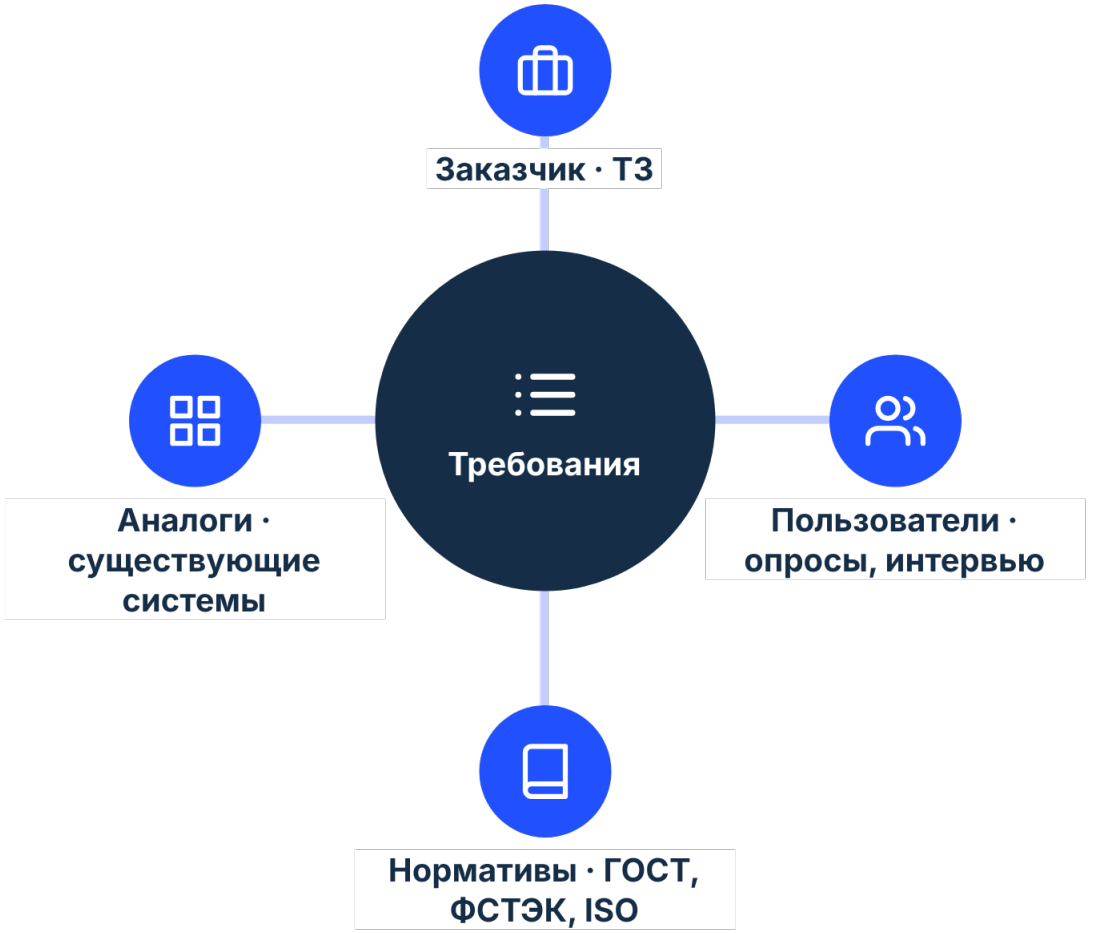
Пользовательские

задачи, которые пользователи смогут решать

Источники требований

Требования собирают из нескольких источников.

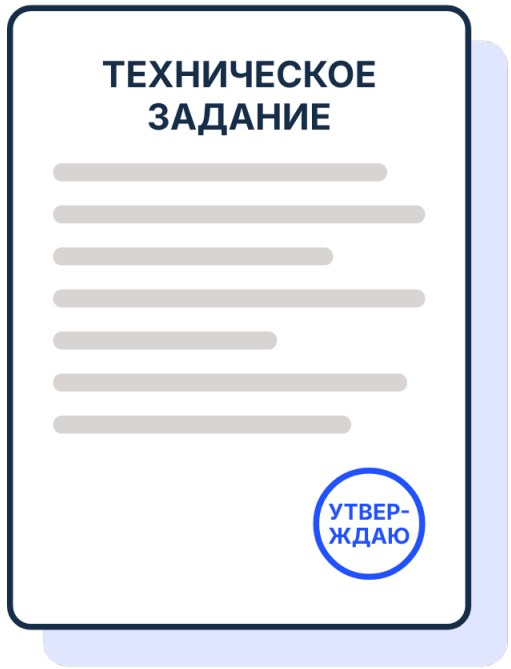
Для ИБ особенно важны **нормативные документы**: ГОСТ, ФСТЭК, ISO



Техническое задание (ТЗ)

ТЗ — официальный документ, определяющий требования к системе, сроки и этапы разработки.

В России оформляется по ГОСТ



- **Требования к системе**
- **Сроки**
- **Этапы разработки**

ГОСТ 34.602-89
автоматизированные системы

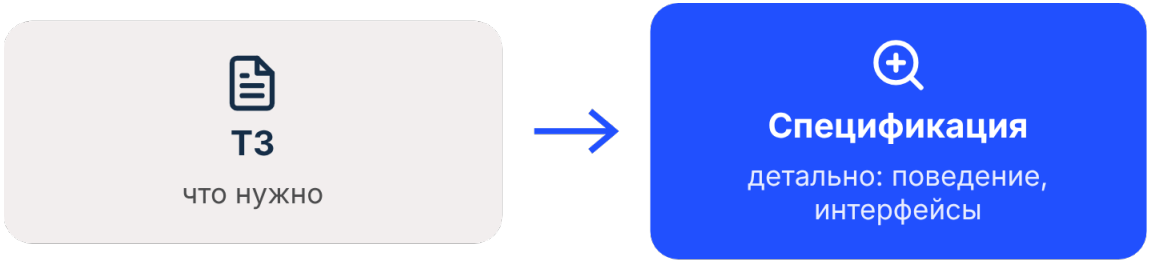
ГОСТ 19.201-78
программные документы

Структура ТЗ по ГОСТ 34.602-89

1. Общие сведения
2. Назначение и цели создания
3. Характеристика объектов автоматизации
4. Требования к системе
   - в целом
   - к функциям
   - к видам обеспечения
5. Состав и содержание работ
6. Порядок контроля и приёмки
7. Подготовка объекта к вводу в действие
8. Требования к документированию
9. Источники разработки

Спецификация программы

Спецификация — более детальный документ: поведение программы, её интерфейсы и ограничения.



Формы спецификации:

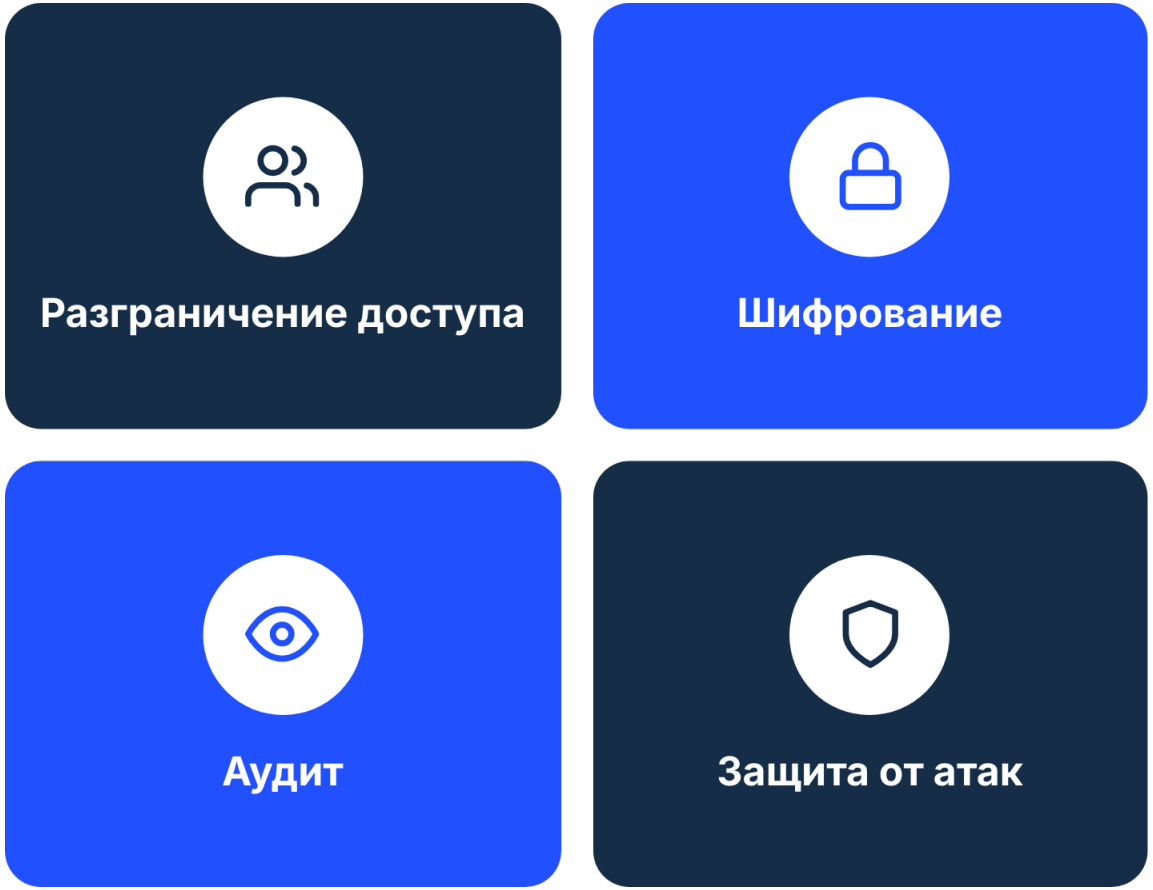
SRS
функциональные и нефункциональные требования

Спецификация API
интерфейсы программных модулей

Спецификация данных
форматы входных и выходных данных

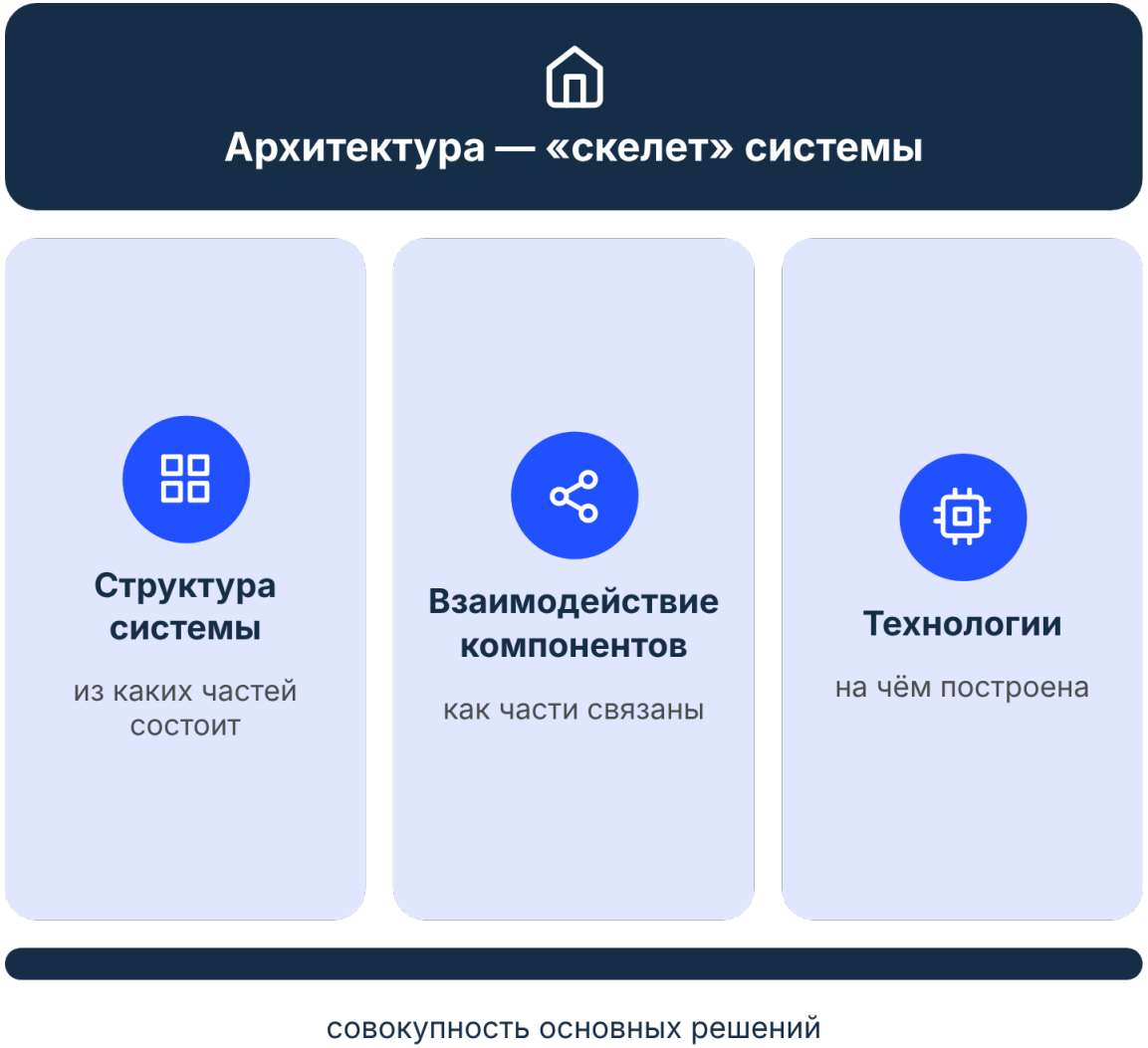
Спецификация и защита информации

Для ИБ спецификация обязательно включает требования к защите информации.

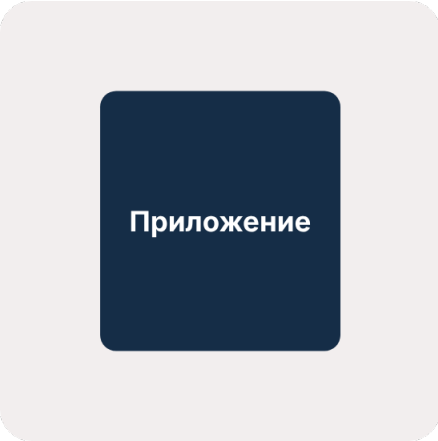


Архитектурное проектирование

Архитектура ПО — совокупность основных решений о структуре системы, взаимодействии компонентов и технологиях.

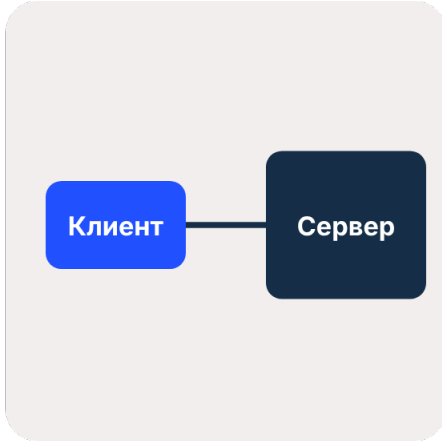


Виды архитектур



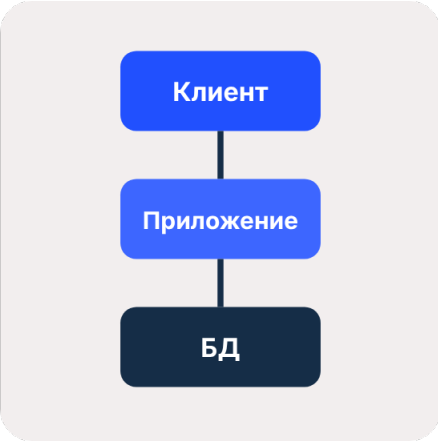
Монолитная

вся система — одно приложение



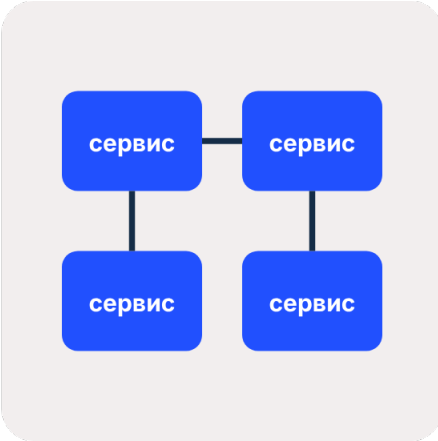
Клиент-серверная

клиентская и серверная части



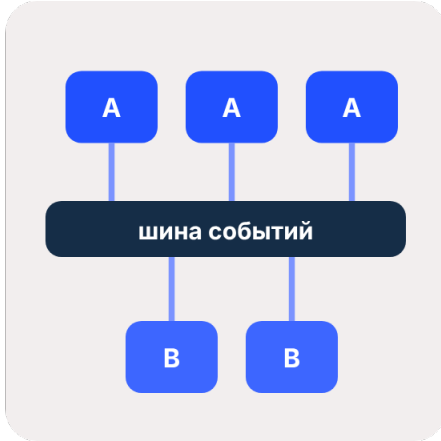
Трёхуровневая

клиент, сервер приложений, БД



Микросервисная

множество независимых сервисов

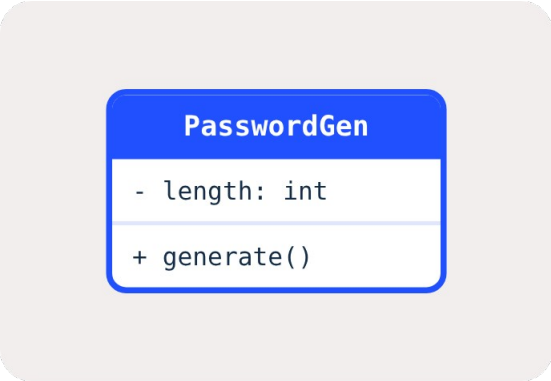


Событийная

компоненты общаются через события

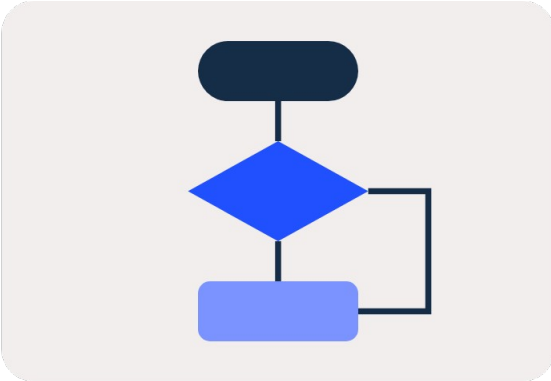
Детальное проектирование

На этом этапе определяются четыре вещи.



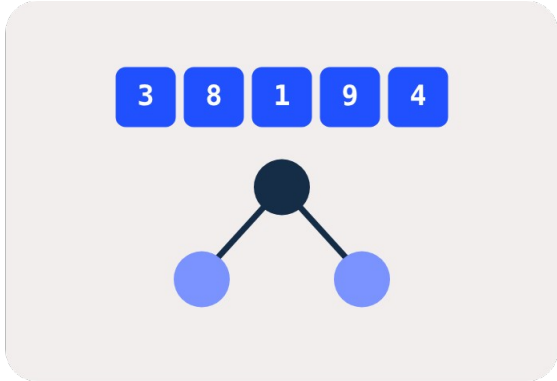
Модули и классы

ответственность, интерфейсы, атрибуты



Алгоритмы

методы решения подзадач



Структуры данных

хранение и обработка данных

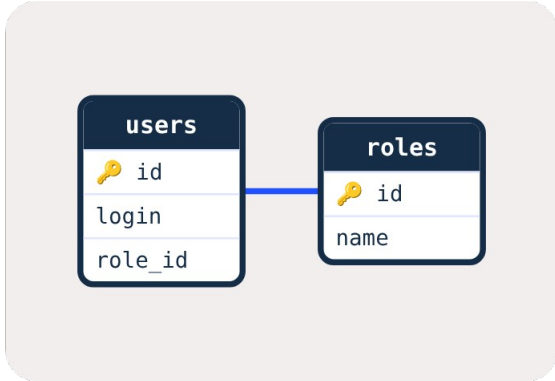


Схема базы данных

таблицы, связи, индексы

Принципы проектирования: SOLID



Single Responsibility

единственная ответственность

Open/Closed

открытость / закрытость

Liskov Substitution

подстановка Лисков

Interface Segregation

разделение интерфейсов

Dependency Inversion

инверсия зависимостей

DRY, KISS, YAGNI

Ещё три принципа, которые делают код проще и надёжнее.

DRY

`Don't Repeat Yourself`

избегайте дублирования кода

KISS

`Keep It Simple, Stupid`

простота — залог надёжности

YAGNI

`You Ain't Gonna Need It`

не добавляйте то, что не нужно сейчас

UML-моделирование

UML — стандартный язык визуального моделирования ПО.

Структурные — из чего состоит

- Диаграмма классов
- Диаграмма компонентов
- Диаграмма пакетов
- Диаграмма развёртывания

Поведенческие — как работает

- Вариантов использования
- Последовательности
- Коммуникации
- Состояний
- Деятельности

UML и информационная безопасность

UML-диаграммы помогают выявить уязвимости ещё на этапе проектирования.

Диаграмма развёртывания показывает физическое размещение компонентов

Это важно для оценки **периметра безопасности**

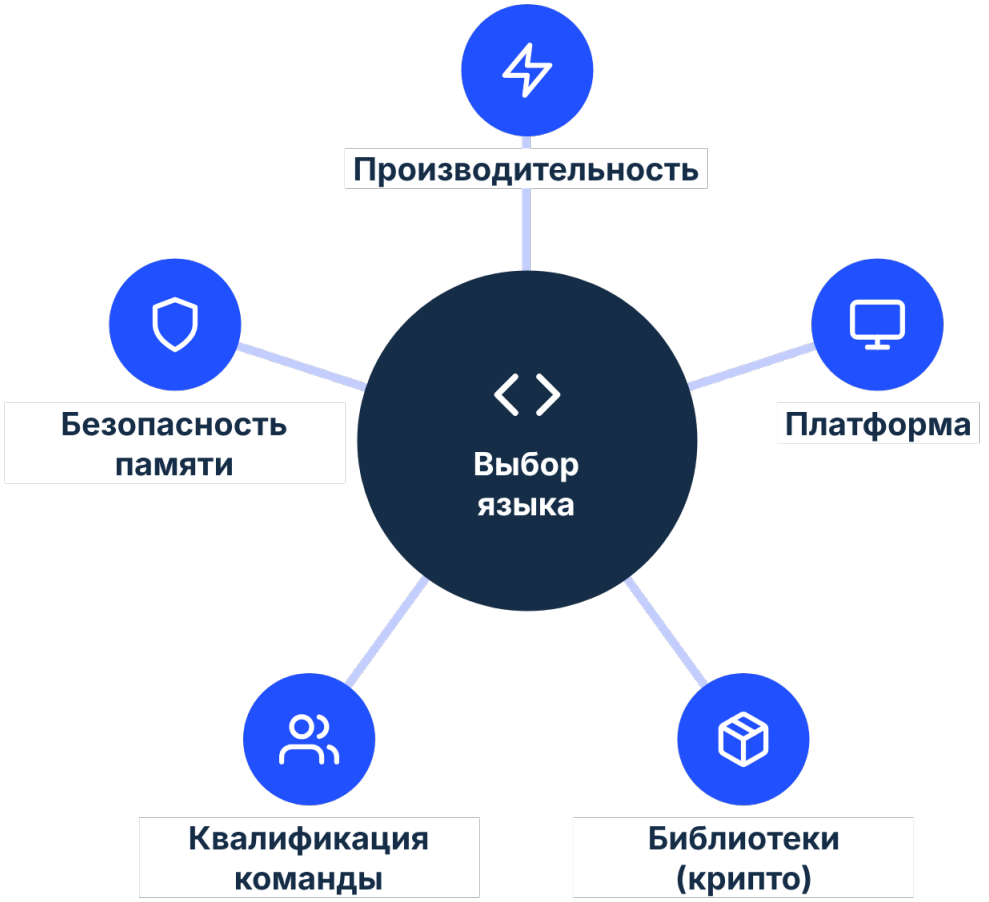


диаграмма развёртывания: где физически размещены компоненты

Выбор языка программирования

Выбор языка зависит от нескольких факторов.

Языки с автоматическим управлением памятью **снижают риск утечек**



Стиль программирования

Совокупность правил оформления кода. Хороший стиль делает код читаемым и сопровождаемым.

Осмысленные имена

Комментарии к сложным участкам

Единообразные отступы, короткие строки, логические блоки

Трудно читать

```
def f(a,b):return a*b if a>0 else 0
```

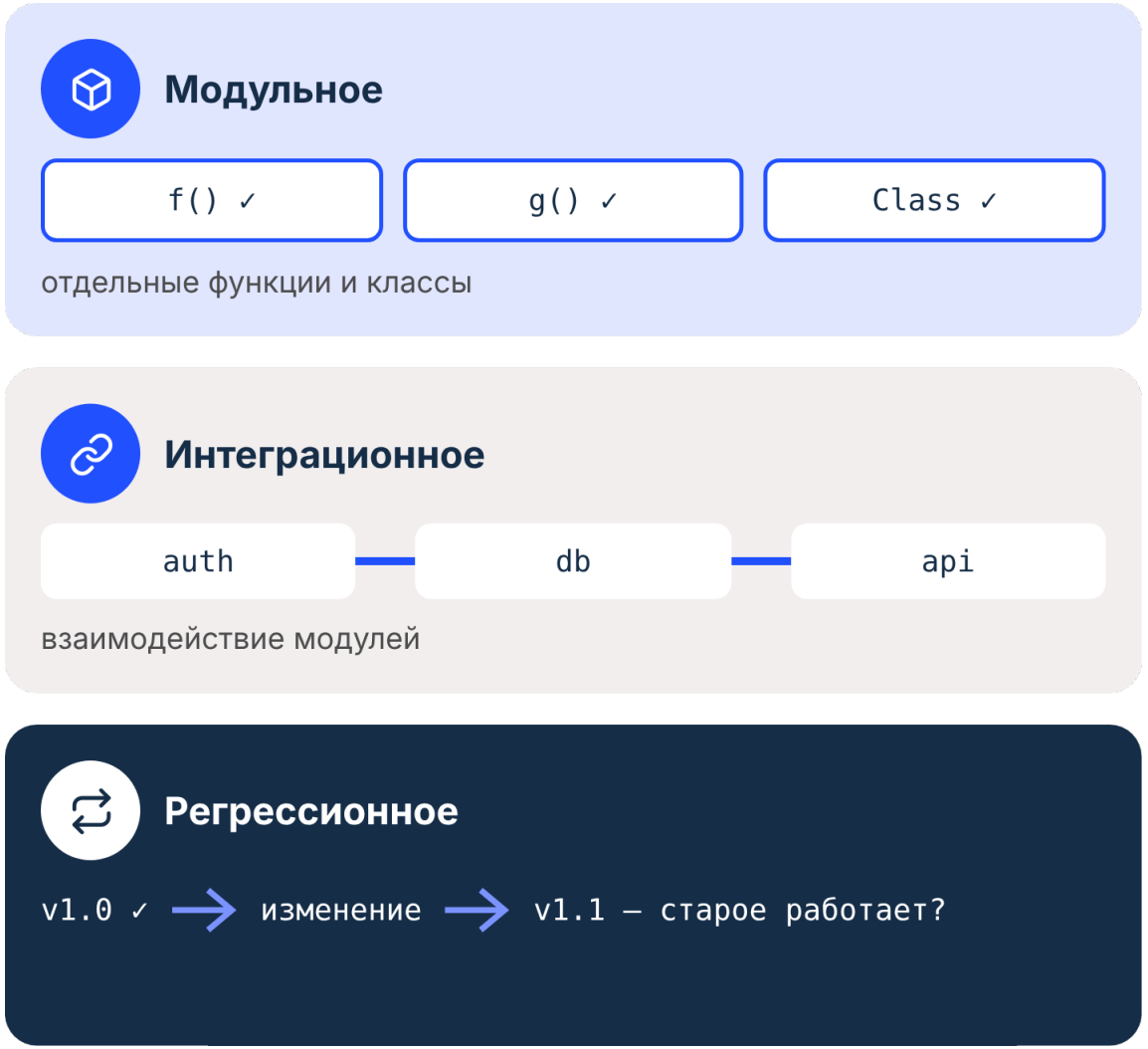
Читаемо

```
def calc_total(price, quantity):
    # отрицательная цена — нулевой итог
    if price <= 0:
        return 0
    return price * quantity
```

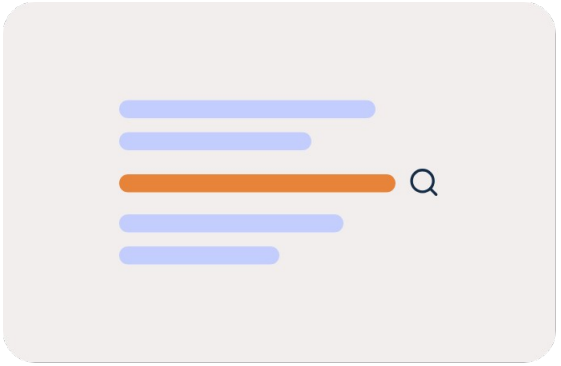
PEP 8 · Python | Google Style Guide | MISRA · C/C++

Тестирование в процессе реализации

Код проверяют на трёх уровнях.



Тестирование безопасности



Статический анализ

код проверяется без выполнения



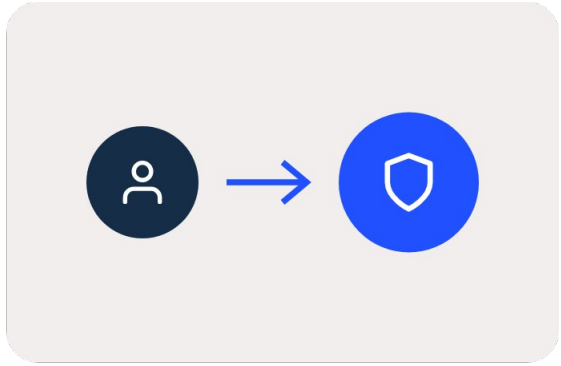
Динамический анализ

поиск уязвимостей во время работы



Фаззинг

подача случайных данных



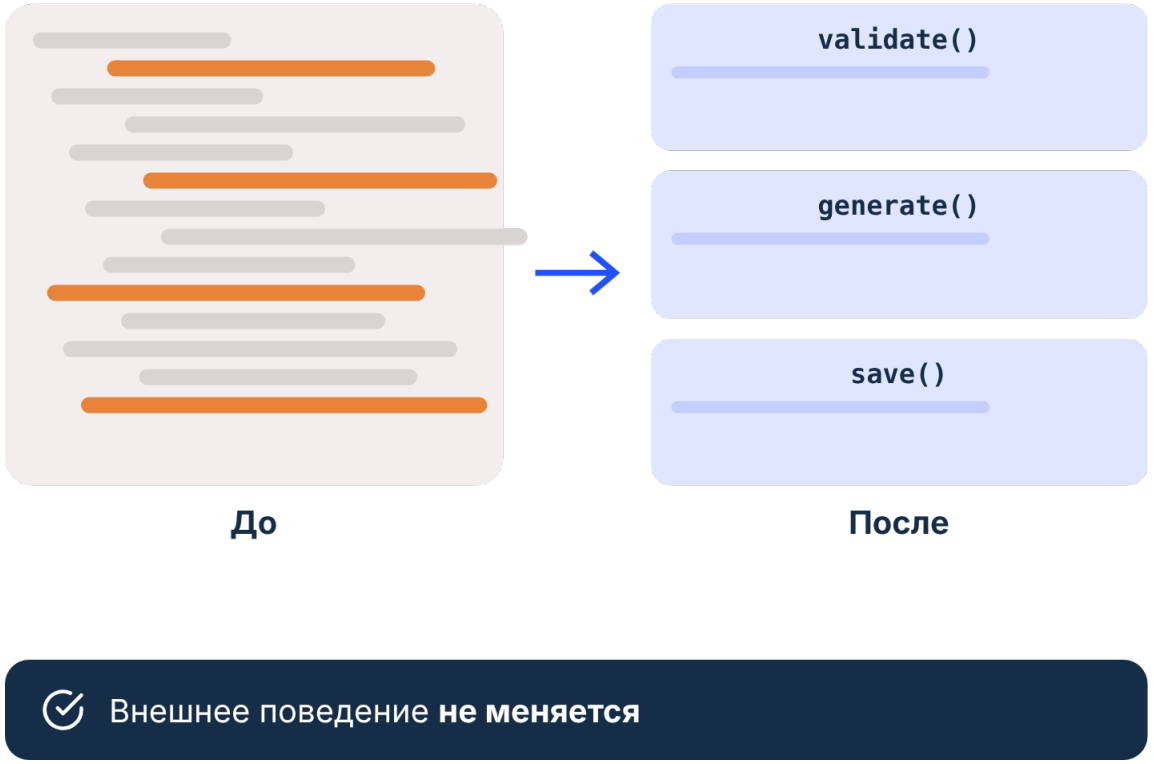
Пентест

имитация атак злоумышленника

Рефакторинг

Рефакторинг — улучшение структуры кода без изменения его внешнего поведения.

Цель: читаемость, меньше сложности, устранение «запахов» кода



Типичные «запахи» кода

Признаки того, что код пора улучшать.

Рефакторинг может как улучшить, так и **ухудшить безопасность**

- **Длинные методы**
- **Большие классы**
- **Дублирование кода**
- **Длинные списки параметров**
- **Избыточные комментарии**

⚠ **Для ИБ:** рефакторинг — только с полным регрессионным тестированием

Виды документации

Без документации невозможны сопровождение, аудит и сертификация.

Пользовательская

- Руководство пользователя
- Руководство администратора
- Справочная система

Разработческая

- ТЗ и спецификация
- Архитектура
- Документация API
- Комментарии в коде

Эксплуатационная

- Установка
- Настройка
- Обслуживание

Управленческая

- План проекта
- Отчёты о ходе работ
- Протоколы совещаний

Стандарты ЕСПД

В России действует Единая система программной документации.

ГОСТ **19.101-77** Виды программ и документов	**ГОСТ 19.201-78** Техническое задание	**ГОСТ 19.202-78** Спецификация	ГОСТ **19.301-79** Программа и методика испытаний	ГОСТ **19.401-78** Текст программы
ГОСТ **19.402-78** Описание программы	ГОСТ **19.403-79** Ведомость держателей подлинников	ГОСТ **19.404-79** Пояснительная записка	ГОСТ **19.501-78** Формуляр	ГОСТ **19.502-78** Описание применения
ГОСТ **19.503-79** Руководство системного программиста	ГОСТ **19.504-79** Руководство программиста	ГОСТ **19.505-79** Руководство оператора	ГОСТ **19.506-79** Описание языка	ГОСТ **19.507-79** Ведомость эксплуатационных документов

Документирование в коде

Комментарии поясняют сложные участки, но не дублируют код

Docstrings описывают модуль, класс, функцию

Автодокументирование — генерация из комментариев

passwords.py

```python
def generate_password(length=16):
    """Генерирует криптостойкий пароль.

    length: длина от 8 до 64 символов.
    Возвращает строку пароля.
    """
    # secrets вместо random: стойкость
    return "".join(secrets.choice(ABC)
                   for _ in range(length))
```

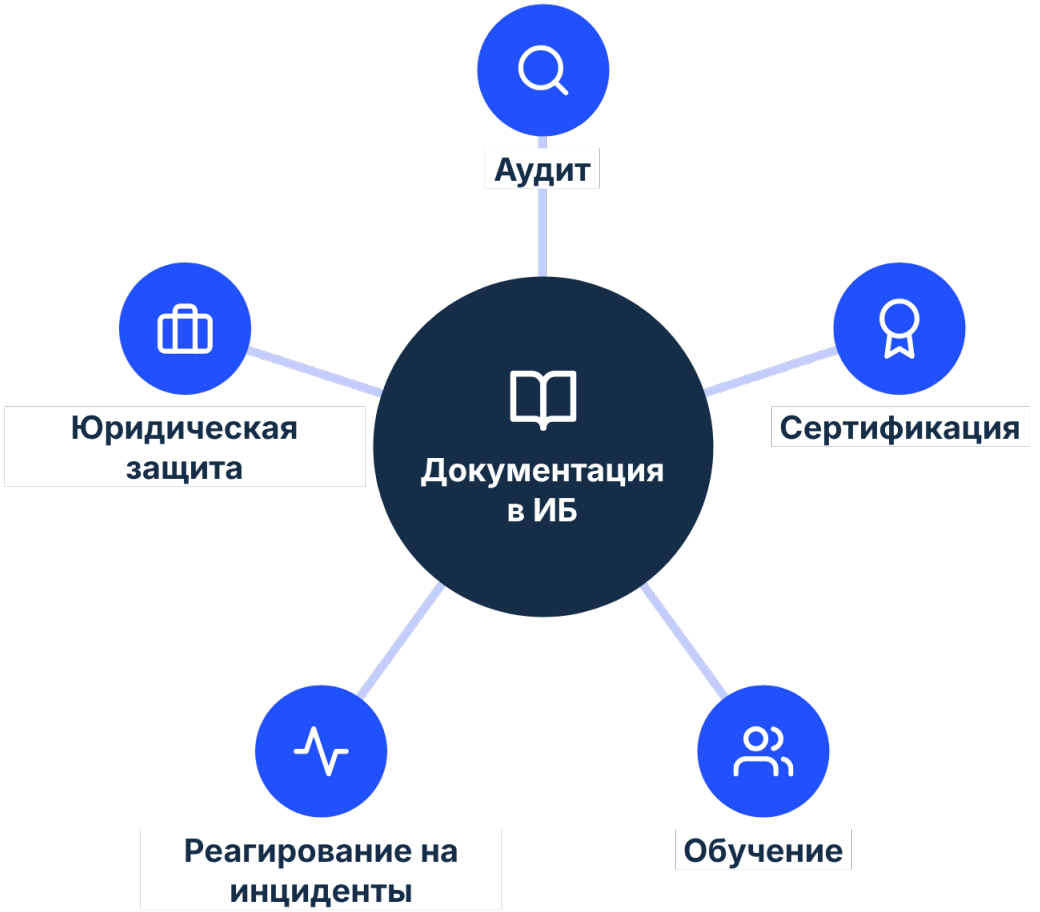
Docstring Комментарий

Sphinx Doxygen Javadoc → HTML / PDF

Документация и информационная безопасность

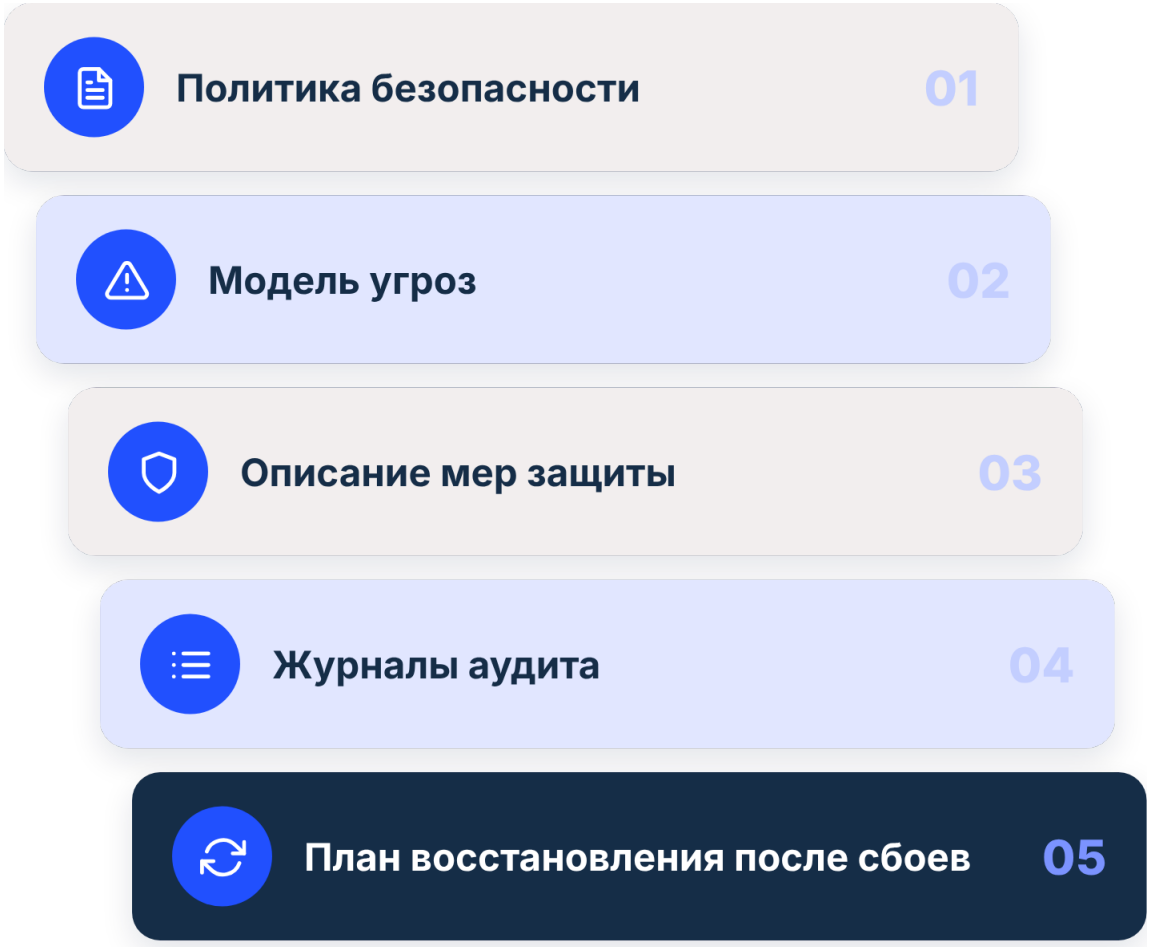
Для ИБ документация выполняет пять функций.

Обязательное условие сертификации **ФСТЭК** и **ISO 15408**



Документация по безопасности

Обязательный набор документов по безопасности системы.



Заключение

Постановка задачи, проектирование, реализация и документирование — фундамент качественного программного продукта.

Безопасность закладывается **с самого начала**

Документация — инструмент доказательства соответствия требованиям

